

三函数 SQL 执行耗时定位与优化分析

生成时间：2026-07-21 22:19

分析对象：fmdb_udf_schmatch 容器中通过 Spark SQL 执行的 F_DW_DETCOLLECT、F_DW_DETTRAIN、F_DW_DETRUN 三个函数。

执行约束：只使用 SQL 方式执行函数，不使用 app 提交。

一、数据来源

本次分析读取了以下来源：

1. 工程落盘输出：
 - doc/20260721/person-info-full-sql-rerun-202607212127/outputs/collect_full_sql.out
 - doc/20260721/person-info-full-sql-rerun-202607212127/outputs/train_full_sql_success_try.out
 - doc/20260721/person-info-full-sql-rerun-202607212127/outputs/detect_full_sql.out
2. 容器内 Hive/Spark SQL 任务表：
 - dw.raha_job_run
 - dw.raha_job_stage_attempt
 - dw.raha_model_artifact
3. 工程源码：
 - RahaJobOrchestrator
 - BoundedParallelExecutor
 - RahaTrainService
 - RahaDetectService
 - raha-defaults.properties

补充说明：容器内未找到可直接复原本次本地 Spark SQL 应用的 Spark event log 或 history 事件目录，因此 Spark 原生 task 级别耗时无法从 History Server 还原。本次精确耗时以 Raha 自身任务表和 SQL 输出为准。

二、三次函数执行总览

函数	jobId	任务类型	SQL 外层耗时	Raha job 耗时	核心服务耗时	最终状态
F_DW_DETCOLLECT	job-12d54e69-f397-46fd-a348-16345c954335	SAMPLING	261.868 秒	248.413 秒	50 毫秒	SUCCESS
F_DW_DETTRAIN	job-43d61f46-cee7-43b0-ae80-1202caebe51c	TRAINING	354.936 秒	333.851 秒	24.714 秒	SUCCESS，内部任务为 PARTIAL_SUCCESS
F_DW_DETRUN	job-ac1bf372-dfbb-49dd-bb21-160bc3c47abe	DETECTION	114.879 秒	97.124 秒	4.664 秒	SUCCESS，内部任务为 PARTIAL_SUCCESS

关键结论：SQL 外层耗时并不等于算法耗时。Raha 内部真正的采样、训练、预测服务耗时分别只有 0.05 秒、24.714 秒、4.664 秒；大量时间花在公共流水线阶段、Spark 作业启动、Hive 元数据读写、分区写入、报表和 ZIP 产物生成上。

三、采样函数耗时拆解

3.1 基本信息

项	值
函数	F_DW_DETCOLLECT
输入 SQL	select * from dw.person_info limit 400
labelingBudget	300
samplingRound	4
sampleBatchId	sample_dw.person_info@20260721134516.455
候选行数	400
采样记录数	300
聚类数	12
聚类字段数	6

3.2 阶段耗时

阶段	开始时间	完成时间	耗时	占 SQL 外层 耗时
LOAD_DATA	2026-07-21 13:41:19	2026-07-21 13:41:28	8.786 秒	3.36%
PROFILE	2026-07-21 13:41:32	2026-07-21 13:41:47	14.415 秒	5.50%
GENERATE_STRATEGY	2026-07-21 13:41:50	2026-07-21 13:41:52	1.332 秒	0.51%
RUN_STRATEGY	2026-07-21 13:41:55	2026-07-21 13:42:27	31.786 秒	12.14%
GENERATE_FEATURE	2026-07-21 13:42:29	2026-07-21 13:42:32	2.304 秒	0.88%
CLUSTER	2026-07-21 13:42:35	2026-07-21 13:45:12	157.315 秒	60.07%
SAMPLE	2026-07-21 13:45:15	2026-07-21 13:45:18	3.430 秒	1.31%
PERSIST_RESULT	2026-07-21 13:45:21	2026-07-21 13:45:23	1.477 秒	0.56%

3.3 采样耗时结论

采样最大瓶颈是 `CLUSTER`，单阶段 157.315 秒，占 SQL 外层耗时约 60.07%。第二瓶颈是 `RUN_STRATEGY`，耗时 31.786 秒，占约 12.14%。实际生成采样任务的服务摘要显示 `elapsedMillis=50`，说明最终选择 300 条样本并不是慢点，慢点在采样前的列画像、策略执行、特征生成和聚类。

采样阶段累计耗时 220.845 秒，Raha job 总耗时 248.413 秒，中间有约 27.568 秒用于阶段状态写入、Hive 元数据访问、分区检查、阶段切换和结果持久化。SQL 外层比 job 多 13.455 秒，主要是 Spark SQL 启动、函数注册、计划编译、UDF 常量折叠和结果取回开销。

四、训练函数耗时拆解

4.1 基本信息

项	值
函数	<code>F_DW_DETTRAIN</code>
输入 SQL	<code>select * from dw.person_info limit 400</code>
使用采样批次	<code>sample_dw.person_info@20260721134516.455</code>
标注批次	<code>ann_sample_dw.person_info_20260721134516.455_sql</code>
标注记录数	300
有效标注数	300
模型集合版本	<code>dw.person_info@20260721135853.933-job-43d61f46-cee7-43b0-ae80-1202caebe51c</code>
策略计划版本	<code>0fa6cc8efde1b903440a99b8299f0795</code>
策略计划数	36
策略族分布	<code>OD=12, PVD=24</code>
策略命中数	2346
特征行数	2400
标签传播数	301
最大字段并发	4

4.2 阶段耗时

阶段	开始时间	完成时间	耗时	占 SQL 外层 耗时
LOAD_DATA	2026-07-21 13:53:54	2026-07-21 13:54:00	6.081 秒	1.71%
MERGE_TRAINING_INPUT	2026-07-21 13:54:05	2026-07-21 13:54:30	25.514 秒	7.19%
PROFILE	2026-07-21 13:54:35	2026-07-21 13:54:49	13.757 秒	3.88%
GENERATE_STRATEGY	2026-07-21 13:54:53	2026-07-21 13:54:55	1.680 秒	0.47%
RUN_STRATEGY	2026-07-21 13:54:58	2026-07-21 13:55:46	48.196 秒	13.58%
GENERATE_FEATURE	2026-07-21 13:55:50	2026-07-21 13:55:52	2.636 秒	0.74%
CLUSTER	2026-07-21 13:55:56	2026-07-21 13:58:39	163.545 秒	46.08%
LABEL	2026-07-21 13:58:43	2026-07-21 13:58:44	1.033 秒	0.29%
PROPAGATE	2026-07-21 13:58:48	2026-07-21 13:58:49	1.197 秒	0.34%
TRAIN	2026-07-21 13:58:52	2026-07-21 13:59:18	25.848 秒	7.28%
PERSIST_RESULT	2026-07-21 13:59:22	2026-07-21 13:59:24	2.572 秒	0.72%

4.3 字段模型结果

字段	状态	分类器	特征维度	训练样本	正例数	正例比例	发布时间
age	PUBLISHED	LOGISTIC_REGRESSION	13	308	25	8.12%	2026-07-21 13:59:08
email	PUBLISHED	LOGISTIC_REGRESSION	12	305	32	10.49%	2026-07-21 13:59:18
id_card	PUBLISHED	LOGISTIC_REGRESSION	16	306	31	10.13%	2026-07-21 13:59:12
name	PUBLISHED	LOGISTIC_REGRESSION	20	300	16	5.33%	2026-07-21 13:59:13
phone	FAILED	无	无	无	无	无	无
home_address	FAILED	无	无	300	4	1.33%	无

4.4 训练耗时结论

训练最大瓶颈仍是 `CLUSTER`，耗时 163.545 秒，占 SQL 外层耗时约 46.08%。第二瓶颈是 `RUN_STRATEGY`，耗时 48.196 秒，占约 13.58%。真正字段模型训练阶段 `TRAIN` 为 25.848 秒，占约 7.28%，并不是最大瓶颈。

训练阶段累计耗时 292.059 秒，Raha job 总耗时 333.851 秒，中间约 41.792 秒消耗在阶段切换、任务表追加写、Hive 分区元数据访问和产物持久化上。SQL 外层比 job 多 21.085 秒，主要是 Spark SQL 启动和 UDF 调用框架开销。

`phone` 字段训练失败表现为 `SparkException`，日志里出现 HDFS 临时 ORC 文件 `FileNotFoundException`，根因更像是并发写 `raha_model_artifact` 时对同一物理表临时目录产生冲突或元数据状态不稳定。`home_address` 字段失败是质量门禁拒绝退化模型，正例只有 4 条，正例比例 1.33%，样本过少导致模型可用性不足。

五、预测函数耗时拆解

5.1 基本信息

项	值
函数	F_DW_DETRUN
输入 SQL	select * from dw.person_info limit 400
模型集合版本	dw.person_info@20260721135853.933-job-43d61f46-cee7-43b0-ae80-1202caebe51c
缺失模型策略	PARTIAL
检测行数	400
字段数	6
有模型字段数	4
失败字段数	2
检测单元格数	1600
检测错误数	134
最大字段并发	4

5.2 阶段耗时

阶段	开始时间	完成时间	耗时	占 SQL 外层 耗时
LOAD_DATA	2026-07-21 14:00:55	2026-07-21 14:01:02	7.645 秒	6.65%
PROFILE	2026-07-21 14:01:09	2026-07-21 14:01:21	12.632 秒	11.00%
GENERATE_STRATEGY	2026-07-21 14:01:27	2026-07-21 14:01:29	1.949 秒	1.70%
RUN_STRATEGY	2026-07-21 14:01:33	2026-07-21 14:02:01	28.273 秒	24.61%
GENERATE_FEATURE	2026-07-21 14:02:06	2026-07-21 14:02:08	2.389 秒	2.08%
PREDICT	2026-07-21 14:02:12	2026-07-21 14:02:19	6.501 秒	5.66%

阶段	开始时间	完成时间	耗时	占 SQL 外层耗时
PERSIST_RESULT	2026-07-21 14:02:24	2026-07-21 14:02:26	2.161 秒	1.88%

5.3 预测耗时结论

预测核心 PREDICT 阶段只有 6.501 秒，真正大头是预测前重复执行的 RUN_STRATEGY、PROFILE、LOAD_DATA 和阶段切换开销。预测阶段累计耗时 61.550 秒，Raha job 总耗时 97.124 秒，阶段之间和任务状态落库有约 35.574 秒。SQL 外层比 job 多 17.755 秒，主要是 Spark SQL 启动和 UDF 框架开销。

预测中 phone 和 home_address 失败不是性能瓶颈，而是训练阶段没有发布对应字段模型。因为 SQL 使用了 missingModelPolicy=PARTIAL，预测仍然完成，最终检测 4 个有模型字段，共 1600 个单元格，发现 134 个错误。

六、耗时到底在哪里

按 SQL 外层总耗时统计：

类别	采样	训练	预测
SQL 外层总耗时	261.868 秒	354.936 秒	114.879 秒
Raha job 内耗时	248.413 秒	333.851 秒	97.124 秒
阶段累计耗时	220.845 秒	292.059 秒	61.550 秒
阶段间和持久化调度开销	27.568 秒	41.792 秒	35.574 秒
SQL 启动和 UDF 框架开销	13.455 秒	21.085 秒	17.755 秒

瓶颈排名：

1. CLUSTER：采样 157.315 秒，训练 163.545 秒，是当前最大耗时来源。
2. RUN_STRATEGY：采样 31.786 秒，训练 48.196 秒，预测 28.273 秒，是第二大耗时来源。
3. 阶段间开销：每次阶段完成都会写 raha_job_run、raha_job_stage_attempt，并触发 Hive 分区和 HDFS 目录检查，三次任务分别消耗约 27.568 秒、41.792 秒、35.574 秒。
4. Spark SQL 进程启动和 UDF 框架开销：三次分别约 13.455 秒、21.085 秒、17.755 秒。
5. 真正模型训练和预测并不是最大瓶颈：训练 25.848 秒，预测 6.501 秒。

七、为什么会这么慢

7.1 每次函数都重复跑公共流水线

采样、训练、预测都重新执行了 LOAD_DATA、PROFILE、GENERATE_STRATEGY、RUN_STRATEGY、GENERATE_FEATURE 等公共阶段。对于同一个 dw.person_info limit 400、同一个 schema 和同一个数据快照，这些中间结果理论上可以复用，但当前 SQL 调用每次都重新生成。

7.2 聚类阶段实现成本高

CLUSTER 对 6 个字段、约 2400 个单元格特征执行字段级聚类。当前默认配置：

配置	值
<code>raha.clustering.distance-metric</code>	<code>COSINE</code>
<code>raha.clustering.target-cluster-count</code>	2
<code>raha.clustering.max-sample-count</code>	500
<code>raha.resource.max-parallel-columns</code>	4

即使数据只有 400 行，聚类仍然占用 2 分半以上，说明这里更可能受算法实现、Spark action 次数、列级并发与本地 `local[2]` 资源不匹配影响，而不是单纯数据量过大。

7.3 策略执行被采样、训练、预测重复触发

策略配置默认启用 `OD, PVD`，本次训练生成 36 个策略计划，分布为 `OD=12, PVD=24`。采样、训练、预测都经历 `RUN_STRATEGY`，导致重复开销：

任务	<code>RUN_STRATEGY</code> 耗时
采样	31.786 秒
训练	48.196 秒
预测	28.273 秒

预测阶段本来只需要使用已发布模型打分，但现在仍然重新跑画像和策略，导致预测前置准备远大于真正预测。

7.4 本地 Spark SQL 启动开销固定存在

三次都是独立 `spark-sql --master local[2]` 进程，每次都会启动 `SparkSession`、加载 Hive、注册函数、编译计划和执行一个只返回一行 JSON 的 UDF 查询。这个固定成本在小数据场景下非常明显。

7.5 任务表写入方式放大了 Hive/HDFS 开销

日志反复出现：

```
The directory hdfs://hdfs:9000/user/hive/warehouse/dw.db/raha_job_run/... was not found. was it deleted very recently?
The directory
hdfs://hdfs:9000/user/hive/warehouse/dw.db/raha_job_stage_attempt/... was not found. was it deleted very recently?
```

这些告警来自前面清理过外部表 HDFS 分区目录，任务运行时每个阶段多次追加写任务表和阶段表。因为 `DIRECT_APPEND` 不做幂等合并，`RUNNING` 和完成态各写一行，读取时也能看到重复状态记录。每次追加写都可能触发 Hive metastore 分区查询、连接重试和 HDFS 目录探测，形成阶段间固定开销。

7.6 并发参数和实际执行资源不匹配

代码中的 `BoundedParallelExecutor` 对策略和字段都使用固定线程池，默认：

配置	值
<code>raha.resource.max-parallel-strategies</code>	4
<code>raha.resource.max-parallel-columns</code>	4
Spark master	<code>local[2]</code>

本地 Spark 只有 2 个执行线程，但 Raha 内部列并发和策略并发是 4。对于 Spark action 密集型阶段，这会造成线程争用、作业排队和元数据连接竞争。`phone` 模型训练期间出现 HDFS 临时 ORC 文件找不到，也和并发写同一张表、同一 HDFS 临时提交目录的稳定性风险有关。

八、优化解决方案

8.1 最高优先级：复用采样与训练中间产物

目标：避免训练和预测重复执行 `PROFILE`、`GENERATE_STRATEGY`、`RUN_STRATEGY`、`GENERATE_FEATURE`、`CLUSTER`。

建议：

1. 以 `datasetId + schemaHash + snapshotId + configversion` 为键缓存画像、策略计划、策略结果、特征字典和聚类结果。
2. 训练阶段如果传入 `sampleBatchId`，优先读取采样阶段已生成的策略结果、特征和聚类，而不是重新计算。
3. 预测阶段只加载模型需要的特征字典和模型，不再重新跑采样用的聚类。

预期收益：采样后训练可少跑约 229 秒以上的重复准备阶段；预测可少跑约 53 秒以上的准备阶段。小数据场景下端到端耗时可能从 12 分钟级降到 3 到 5 分钟级，甚至更低。

8.2 优化 `CLUSTER` 阶段

目标：解决最大单点瓶颈。

建议：

1. 对 400 行以内的小数据，增加轻量聚类路径，避免调用完整 Spark 聚类或重复 action。
2. 将 `raha.clustering.max-sample-count` 从 500 下调到与业务标注预算匹配，例如 300 或 200。
3. 对采样和训练复用同一份聚类结果，避免同一批数据聚类两次。
4. 对文本类字段可以先使用策略覆盖分数直接采样，聚类只作为补充，而不是必经路径。
5. 在阶段日志中补充每列聚类耗时和样本量，确认是否某个字段拖慢整体。

预期收益：当前采样和训练聚类合计 320.860 秒，是最值得优化的地方。

8.3 优化 `RUN_STRATEGY` 阶段

目标：减少重复策略计算和无效策略数量。

建议：

1. 预测阶段默认不要重新执行策略，只使用训练时发布的模型和特征字典。
2. 开启策略过滤或指定字段白名单，避免对不需要预测或没有模型的字段继续跑策略。

3. 对 `person_info` 这类结构化字段先保留收益最高的策略类型，例如优先保留 PVD 格式、长度、字符集策略，对纯数值列保留必要 OD 策略。
4. 将策略运行结果持久化并按快照复用。

预期收益：三次任务 `RUN_STRATEGY` 合计 108.255 秒，复用后可显著下降。

8.4 降低任务表和阶段表写入成本

目标：减少阶段间 27 到 42 秒的固定开销。

建议：

1. 把 `raha_job_run`、`raha_job_stage_attempt` 从追加状态流水表改为快照表或支持按 job/stage 覆盖最新状态。
2. 每个阶段只在开始和结束写必要字段，减少中间 `jobRepository.save` 调用。
3. 分区目录清理后立即执行 `MSCK REPAIR TABLE` 或按需预创建当月分区目录。
4. 对任务表写入使用批量写或延迟汇总写，阶段结束后一次性写完整状态。
5. 对小数据验证场景，可以临时关闭非必要列产物持久化，保留最终结果表即可。

预期收益：每次任务可减少十几到几十秒的 Hive/HDFS 元数据开销。

8.5 调整本地验证并发

目标：让 Raha 内部并发与 `local[2]` 匹配。

建议：

1. 本地 SQL 验证时设置 `raha.resource.max-parallel-columns=2`。
2. 本地 SQL 验证时设置 `raha.resource.max-parallel-strategies=2`。
3. 如果使用真实 Spark 集群，再根据 executor core 数把并发调高。
4. 对模型发布写入同一张表的阶段，先串行发布或按字段分区写，避免 HDFS 临时目录竞争。

预期收益：不一定降低所有阶段耗时，但能减少并发写冲突和 Spark 作业争用，提升稳定性。

8.6 优化 SQL 调用方式

目标：减少每次独立 `spark-sql` 进程的固定启动成本。

建议：

1. 如果仍要求只能用 SQL，可以使用同一个 `spark-sql` 会话顺序执行采样、标注插入、训练和预测，避免三次独立启动。
2. 将函数注册和固定配置放到初始化 SQL 中，减少每次执行前的准备语句。
3. 保留 SQL 调用，不使用 app 提交，但可以用同一 SQL session 执行多条语句。

预期收益：三次固定 SQL 外层开销合计约 52.295 秒，可明显减少。

8.7 改善可观测性

目标：以后能直接定位到字段、策略和 Spark action 级别。

建议：

1. 开启 Spark event log，并挂载持久化目录，例如 `/tmp/spark-events` 或 HDFS 事件目录。
2. 启动 Spark History Server，保留每次 SQL 的 application、job、stage、task 明细。
3. 在 Raha 阶段表 `summary_json` 中写入每列耗时、每个策略族耗时、Spark action 次数和结果行数。
4. 在 `BoundedParallelExecutor` 中记录每个 workKey 的开始、结束、耗时和线程等待时间。

5. 对 `sparkSqlFmdbContextGateway.append` 增加耗时日志，拆分为 metastore 查询、DataFrame 写入、分区刷新三个子耗时。

九、最终结论

目前耗时主要不在最终采样选择、模型训练或预测打分本身，而在三类地方：

1. 重复公共流水线，尤其是 `CLUSTER` 和 `RUN_STRATEGY`。
2. 小数据场景下 Spark SQL 独立进程启动和 UDF 执行框架固定成本偏高。
3. 每个阶段频繁追加写 Hive/HDFS 任务表，叠加外部表分区目录曾被清理后的元数据探测和连接重试。

最优先的优化路线是：先让训练复用采样的画像、策略、特征、聚类结果；再让预测跳过采样/训练前置策略流水线，只加载已发布模型和必要特征；最后减少阶段表写入次数并启用 Spark event log。按本次实测，单是避免重复 `CLUSTER` 和 `RUN_STRATEGY`，理论上就能省掉数百秒级耗时，这是当前收益最大的方向。